

JANUS: Enhancing Asynchronous Common Subset with Trusted Hardware

Liangrong Zhao
Monash University

liangrong.zhao@monash.edu

Hans Schmiedel
Monash University

hans.schmiedel@monash.edu

Qin Wang
CSIRO Data61, Australia

qin.wang@data61.csiro.au

Jiangshan Yu*

University of Sydney
jiangshan.yu@sydney.edu.au

Abstract—Asynchronous common subset (ACS) has been extensively studied since the asynchronous Byzantine fault tolerance (BFT) framework was introduced by Ben-Or, Kemler, and Rabin (BKR). The line of work (i.e., HoneyBadgerBFT, BEAT, EPIC) uses parallel reliable broadcast (RBC) and asynchronous binary agreement (ABA) instances to reach an agreement on a subset of proposed transactions.

In this paper, we further progress the BKR paradigm by presenting JANUS, the first hybrid ACS protocol leveraging trusted hardware components. JANUS is the first ACS protocol that tolerates a minority of Byzantine processes and that has $\mathcal{O}(n^2)$ message complexity. Supported by trusted hardware components, we introduce a provable broadcast primitive to replace RBC, and develop a resilient binary agreement protocol. Messages for concurrent instances of agreement are aggregated into vectors. Our experimental results demonstrate significant performance improvements over predominant ACS constructions with a 92%+ increase compared to HoneyBadgerBFT and a 47%+ increase compared to BEAT. Additionally, we provide a comparison with open-source hybrid BFT protocols that operate under a partially synchronous network, highlighting the performance enhancement compared to previous hybrid protocols that also tolerate the Byzantine minority (e.g., MinBFT and Damysus, by 49%+).

Index Terms—Asynchronous Common Subset, BFT, Trusted components, Asynchronous

I. INTRODUCTION

Byzantine fault tolerance (BFT) protocols gained significant attention, especially in the context of blockchain [1]. BFT protocols enable a distributed network of n processes to reach an agreement in the presence of f malicious parties (i.e., Byzantine processes). For applications operating across the Internet, asynchronous BFT protocols that guarantee safety and liveness with minimal network assumptions are preferred [2][3][4].

Although asynchronous BFT protocols are favorable, even necessary, in many practical scenarios, achieving a practical instance is rather challenging. Ever since the first asynchronous agreement protocol was introduced [5], a series of subsequent implementations have either relied on redundant message exchange patterns, resulting in high communication complexity [6][7][8], or suffered from a lack of responsiveness, leading to sluggish runtime [2].

ACS and its constructions. The asynchronous common subset (ACS) is a variant of Byzantine consensus under an asynchronous network, where distributed processes each propose a value via a broadcast protocol as input and reach

an agreement on a common subset of at least $n-f$ input values as output. A secure replicated state machine (such as a blockchain) under an asynchronous network can be implemented using totally ordered atomic broadcast, where ACS serves as a core building block, as demonstrated by HoneyBadgerBFT (ACS with threshold encryption) [2], the first practical asynchronous BFT atomic broadcast. ACS is also frequently used as a foundational primitive for constructing applications in distributed environments, such as distributed key generation [9][10], secret sharing [11], and multi-party computation [12][13] (also reported in [14]).

BKR and alternatives. Pioneering in this field, Ben-Or, Kelmer, and Rabin [15] provided a practical ACS framework (denoted as the BKR paradigm) by incorporating Reliable Broadcast (RBC) instances and Asynchronous Binary Agreement (ABA) instances, inspiring a line of studies such as HoneyBadgerBFT, BEAT [3], and EPIC [4]. PACE proposes a generic framework similar to BKR by replacing the traditional ABA with a novel reproposeable ABA to bias the agreement preference towards one, accelerating the voting procedure in the agreement phase [16].

Cachin, Kusawe, Petzold, and Shoup [6] (a.k.a., the CKPS paradigm) take a different path to reduce ACS to asynchronous multivalued validated Byzantine agreement (MVBA, rather than ABA), followed by a series of works improving on MVBA-based ACS primitives, including FIN [14], Dory [17] and the Dumbo family protocols [18][19][20]. MVBA has been explored for optimal time complexity [6][21][18][20][14][17] to address BKR's performance bottleneck (*OP-I*). However, MVBA-based constructions are only computationally secure and rely on less well-established cryptographic pairing assumptions, as reported by an earlier study [16]. Moreover, in comparison to ABA, MVBA tends to exhibit inferior performance in normal-case scenarios and medium-sized networks. For instance, Speeding Dumbo's sMVBA requires 6 rounds of communication in the best case, while ABA only needs one [22]. This drives us to reconsider the basic ABA design.

Open problems (OP) in BKR. While BKR and its alternatives offer promising avenues for constructing asynchronous BFT protocols, several challenges persist (along with our initial statements above), impeding practical implementation.

OP-I: BKR is constrained by normal-case performance.

*Corresponding author

In the BKR paradigm, each process commonly utilizes reliable broadcast to disseminate a transaction to all other processes, followed by n parallel ABA instances to determine the inclusion of transactions in the final subset. However, a bottleneck arises as it's almost certain that some of the n parallel executions might be delayed due to network conditions or arbitrary behaviors. As demonstrated in latency breakdown analyses from experiments, this bottleneck is even more severe in larger networks, where the majority of the delay stems from the agreement phase [19]. Consequently, the BKR paradigm has received less optimization attention in recent years due to its high message complexity. Particularly, when compared with the MVBA-based Speeding Dumbo [22], which boasts a message complexity of $\mathcal{O}(n^2)$, BKR typically incurs a message complexity of $\mathcal{O}(n^3)$ due to its " n RBC + n ABA" design, even though it has less number of communication steps in normal cases (i.e., not under attacks).

OP-II: BKR overlooked resilience optimization.

While many works [6][21][18][20][14][17][7] focused on reducing the message complexity of asynchronous BFT protocols, there has been a lack of research exploring potential resilience improvements compared to the significant effort on partially synchronous BFT protocols [23][24]. With increasingly complex and untrustworthy environments (e.g., permissionless settings in blockchain systems) and the proliferation of sophisticated attacks (e.g., [25][26]), an improved resilience to tolerating the Byzantine minority is desired.

These problems motivate us to ask the following question:

Can we further improve the BKR construction to reduce message complexity with optimal resilience?

With an affirmative answer, we made a series of efforts.

The road to reduce message complexity. Our focus is to optimize the message complexity of the two fundamental components within the ACS framework, two key phases: *broadcast* phase and *agreement* phase. We aim to enhance the message complexity to achieve an optimal $\mathcal{O}(n^2)$. We first examine the obstacles that hinder BKR's message complexity.

Broadcast phase: provable broadcast & property weakness.

Inspired by VABA [21] and Speeding Dumbo [22], adapting provable broadcast (PB) to replace the expensive RBC is a practical approach to reduce the communication complexity to the optimal in the broadcast phase. However, directly employing PB in the BKR paradigm is unfeasible due to the lack of all-to-all communication, which is essential to deliver the *agreement* property. VABA incorporates four consecutive PB invocations to uphold safety and liveness while Speeding Dumbo introduces two more all-to-all communication rounds in MVBA to maintain only one PB execution for the broadcast phase. Speeding Dumbo also adds a recovery phase at the end of its ACS to address this property loss. To avoid sacrificing the BKR's efficiency in communication rounds in the normal case, we aim to replace the RBC with only one PB invocation without introducing additional message exchange steps.

Agreement phase: high costs of parallel instances.

The message complexity of the agreement phase needs to be optimized to $\mathcal{O}(n^2)$ to prevent it from being the bottleneck of the protocol's message complexity. Since each of the n transactions in ACS requires individual agreement outcomes, n parallel agreement instances are necessary as each ABA instance only produces the outcome for one transaction. It is rather challenging to reduce the message complexity of a single ABA to $\mathcal{O}(n)$ as we believe all-to-all communication is necessary for an agreement protocol.

Tools to clear the obstacles. We utilize architecture hybridization to construct JANUS¹, a practical ACS primitive with trusted hardware assistance. Architecture hybridization, which leverages trusted hardware components [27], is known to offer enhanced functionalities (e.g., confidentiality [28]) or trustworthiness (e.g., resilience [24][29]) of the overall system. While we acknowledge that vulnerabilities may exist in the commercial off-the-shelf trusted components, such as Intel Software Guard Extensions (SGX), our trusted services are generic enough to be potentially implemented in any trusted execution environment (TEE).

Existing efforts with architecture hybridization have primarily used trusted hardware components to achieve non-equivocation so that each replica commits to a single order for each request by having the trusted component certify each message it broadcasts [24][23][30][31]. However, most aforementioned trusted protocols operate in synchrony or partial synchrony. To our knowledge, no attempts have been made to construct ACS by leveraging trusted hardware. However, a trivial adaptation to simply apply the non-equivocation configuration of existing partial synchronous protocols cannot work in asynchronous settings, as the instance confronts new technical challenges (TCs) stated as follows.

TC-I: Unbounded message delays.

Current trusted protocols exhibit inherent *sequentiality* [32], where the primary process has to certify each message it disseminates, typically with a monotonic counter number as a unique identifier, or to log the message in an append-only fashion [33][23][24]. This sequential feature presents a challenge for asynchronous protocols where processes cannot expect a message to be delivered within a bounded delay, potentially leading to the need for unbounded secure memory. Pipelined protocols are explored to overlap the various execution steps of different transactions, enabling concurrent processing [34][24][35]. However, hybrid (namely, using trusted hardware) pipelined protocols require transactions to be arranged in a strict gapless sequence, which is still subject to the challenge of unbounded delay in asynchronous settings [24][29][35].

TC-II: Parallel session management.

Most trusted hardware-assisted BFT protocols [30][36][29] operate on a leader-based model, where only a single execution is disseminated and processed across the network. Consensus-oriented parallelization is also explored in trusted protocols

¹JANUS, named after the Roman god of gates and passages, which requires messages to pass through trusted components.

to achieve full parallelism by separating the agreement process into multiple partitions before merging the consensus outcomes into one [30][36]. However, such designs typically require non-lightweight hardware, as each process needs to equip multiple hardware components to prevent threads of different partitions from racing for trusted hardware access.

Our solution. We utilize trusted hardware to implement two critical components: the *counter* and the *collector*. The counter assigns sequence numbers to outgoing transactions broadcast by the host process, ensuring order and consistency. The collector stores the interim states of transactions during the broadcast and agreement phases, restricting arbitrary access to these stored states to prevent faulty processes from making conflicting or malicious alterations. Given the size of the network n , only a constant size list is stored inside the collector instead of requiring ever-growing secure memory storage (addressing *TC-I*). Additionally, our setup for parallelisation is lightweight, as each process only needs one counter, in contrast to existing work requiring multiple counters per process [30] or complex design [35] (addressing *TC-II*).

Our contributions. With the counter and collector, we can solve the aforementioned challenges and propose JANUS, the first trusted hardware-assisted ACS protocol that incorporates provable broadcast (PB) and an agreement protocol with aggregated messages. We highlight our contributions as follows.

- *We implement a trusted provable broadcast tailored for the broadcast phase of ACS, replacing reliable broadcast.* We propose replacing the RBC in the broadcast phase with a more cost-effective PB protocol, which exhibits linear complexity in just three steps (one-all-one-all). This replacement eliminates the need for RBC's all-to-all communication steps, leading to a reduced message complexity of $\mathcal{O}(n)$ for one broadcast instance and $\mathcal{O}(n^2)$ overall. Our trusted provable broadcast protocol also ensures that such complexity reduction does not impact the progression of the following agreement protocol. The utilization of trusted hardware can circumvent the necessity for a strong broadcast property and facilitate the agreement progress by relying on a relaxed property.
- *We implement an agreement protocol with the aggregated messages from independent inputs.* We further develop a merged version of ABA, which maintains a reduced message complexity of $\mathcal{O}(n^2)$. We propose an agreement protocol where for every step in the agreement phase, each of the n processes sends *only one* vote messages to each of the n recipients, resulting in a message complexity of $\mathcal{O}(n^2)$. We propose aggregating a process' n vote messages, each containing a binary vote (namely, $\{0,1\}$) originally intended for n ABA instances, into a single message containing a vector of n binary votes. Each vector element represents an individual vote for a transaction.
- *We provide a formal analysis of the complexity and security of JANUS.*

We provide proofs to demonstrate that JANUS satisfies the core properties of ACS. Our proofs demonstrate that JANUS can tolerate up to f Byzantine faults with a total of $(2f + 1)$ processes. The message complexity is $\mathcal{O}(n)$ per transaction, and the protocol's message complexity with n parallel transactions processes is $\mathcal{O}(n^2)$.

- *We implement JANUS as the first trusted hardware-assisted asynchronous BFT protocol and examine performance gains.*

In our evaluation, we compare JANUS against prominent open-source protocols, including both ACS protocols (e.g., HoneyBadgerBFT [2]) and protocols incorporating trusted components (e.g., MinBFT [23], Damysus [24]). In our geo-distributed setup on the Google Cloud platform with Intel SGX's SDK, JANUS exhibits higher throughput than HoneyBadgerBFT across the tested batch size range from 2000 transactions to 10000 transactions, by an average of **92%**. Additionally, the latency is significantly reduced by an average of **63%**. When compared with other trusted protocols, JANUS demonstrates higher throughput than MinBFT by an average of **89%** and outperforms Damysus for large network sizes up to **72%** thanks to JANUS' fully parallel design.

II. SYSTEM MODEL & PROBLEM STATEMENT

A. System Model

Processes. We consider a network $\Pi = \{P_0, P_1, \dots, P_{n-1}\}$ of n processes with identifiers known to all in the network. We assume that processes are authenticated, i.e., they can rely on a digital signature scheme to authenticate each other's messages. We consider up to $f \leq \lfloor \frac{n-1}{2} \rfloor$ processes may be Byzantine. Byzantine processes may behave arbitrarily, but cryptographic primitives remain secure. In addition, all components of a Byzantine process can be tampered with except the ones providing trusted services.

Asynchronous network. We consider an asynchronous network, where messages sent by correct processes are eventually delivered after an unknown delay. Processes are connected in a distributed manner where each process is able to communicate with any other process via a direct connection. The communication link between each pair of processes ensures that no message from a correct sender is lost, duplicated or indefinitely delayed. However, messages can be arbitrarily delayed or reordered.

Adversary model. JANUS is a hybrid protocol where all processes are equipped with trusted components. At faulty (Byzantine) processes, all components can be tampered with except those providing trusted services. Byzantine processes may behave arbitrarily and may not interact with the trusted component. For example, they may choose never to respond to protocol operations or only respond to some correct processes. An adversary can delay message delivery by any finite amount of time, although messages from correct processes must eventually be delivered, with no bound on delivery time. We assume that public keys are known to all parties (e.g.,

via a PKI) and that all cryptographic algorithms are secure. We acknowledge there are security concerns regarding popular trusted hardware, we provide a detailed security analysis of JANUS under novel attacks such as side-channel attacks and Iago attacks in Appendix C.

B. Problem Statement

Asynchronous common subset. We aim to develop a hybrid asynchronous ACS as a consensus protocol. This can be easily extended into an asynchronous atomic broadcast for implementing BFT replicated state machines [2][6]. In an ACS protocol, each process broadcasts a value (referred to as a transaction) as input, and all correct processes will output a common subset of the inputs, containing transactions from at least $n-f$ processes. Formally, a secure ACS protocol satisfies the following properties [2]:

- **ACS-validity:** If a correct node outputs a set $\mathbb{S}$, then $|\mathbb{S}| \geq n-f$, and the set $\mathbb{S}$ contains the inputs of at least $n-2f$ correct nodes.
- **ACS-agreement:** If a correct node outputs $\mathbb{S}$, then every correct node outputs $\mathbb{S}$.
- **ACS-totality:** If $n-f$ correct nodes receive an input, then all correct nodes produce an output.

ACS protocols generally consist of two phases: (i) *broadcast*: transactions are disseminated to all other processes; and (ii) *agreement*: correct processes reach an agreement regarding which transactions should be included in the final subset. Our ACS construction optimizes both phases (cf. Sec.III) to achieve optimal resilience and reduce message complexity.

Complexity assessment. In this paper, the message complexity refers to the total number of messages generated by correct processes and the communication complexity refers to the total number of bits exchanged among correct processes [6]. We achieve lower message complexity while communication complexity is aligned to other asynchronous BFT protocols.

III. JANUS IN A NUTSHELL

The BKR Paradigm. In a typical BKR paradigm [15], each process operates both an RBC and an ABA instance. Each process acts as a primary (sender) in its own instances while also engaging in all other instances as an ordinary replica. It is assumed that every process submits a transaction to its RBC instance for dissemination. A subsequent ABA instance then makes a binary decision (accept/reject) regarding the inclusion of this transaction in the final subset. A correct process initially votes 1 in an ABA instance only if the corresponding transaction has been delivered to it. All transactions with a positive consensus outcome comprise an ACS such that correct processes obtain the same subset of transactions.

JANUS overview. We propose a trusted ACS protocol JANUS that incorporates parallel provable broadcast instances along with an agreement instance using vector vote messages (structure diagram given in Fig.1). Our protocol has three new contributions regarding the protocol design.

Design-① Increasing the tolerance threshold.

Each process is equipped with an identical hardware configuration, featuring two trusted services: *counter* and *collector*. The counter assigns unique identifiers to transactions, while the *collector* keeps a record of the key internal states and facilitates the corresponding operations. This design ensures that our protocol can *increase the tolerance threshold to malicious minority* ($f \leq \lfloor \frac{n}{2} \rfloor$).

Design-②: Obtaining a linear broadcast instance.

There are n parallel broadcast instances, with each process serving as the sender (referred to as the *primary* from a consensus view) of an instance. In each instance, the primary initiates a transaction by broadcasting a **Propose** message. Processes receiving messages respond with **Endorse** after verification. Once a threshold of **ENDORSE** messages is reached, the primary broadcasts a **Cert** message. Processes receiving **Cert** consider the initial transaction as delivered. This design ensures *linear message complexity per transaction*.

Design-③: Reducing message complexity in agreement.

Departing from the conventional BKR paradigm that runs a separate BA instance for each transaction, processes in JANUS combine the votes for the parallel BA instances into a vector, thereby *reducing the message complexity per transaction from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$* .

A. Broadcast Phase

Prior to commencing BA for binary decisions, transactions have to be disseminated as votes in the agreement phase rely on their delivery status. All protocols in the BKR paradigm employ the well-explored RBC as their broadcast primitive. Bracha's original double-echo RBC protocol results in a total communication overhead of $\mathcal{O}(n^2)$ as it requires rounds of all-to-all communication to satisfy the following properties [37].

- **RBC-validity:** If a correct process p broadcasts a message m , then every correct process eventually delivers m .
- **RBC-none-duplication:** No correct process delivers message m more than once.
- **RBC-integrity:** If a correct process delivers a message m with a correct sender p , then m was broadcast by p .
- **RBC-agreement:** If some correct process delivers a message m , then every correct process eventually delivers m .

With the hardware assistance, we propose to replace the RBC with a more cost-effective PB protocol, which demonstrates linear complexity in only three steps (one-all-one-all). This replacement eliminates RBC's all-to-all communication phases, resulting in a reduction in message complexity. However, in contrast to RBC with all-to-all communication, the non-primary processes in PB are constrained to receiving messages solely from the primary. Consequently, in the event of a faulty primary, not all properties of reliable broadcast can be guaranteed in the PB, even with hardware assistance. Notably, the achievement of **RBC-agreement** becomes unattainable, as there is no assurance that all correct processes will eventually deliver a message when the primary's correctness is not guaranteed [37]. To complement it, our design bypasses the

need for the **RBC-agreement** property not provided by PB and only relies on its **PB-consistency**²:

- **PB-consistency**: If some correct process delivers a message m and another correct process delivers a message m' , then $m = m'$.

Compared with the agreement property, consistency does not guarantee *totality*, ensuring that if a message is delivered to a correct process, every correct process will eventually receive the same message [37]. There might be a situation where one correct process delivers a message while another correct process has not yet received this message from the sender.

VABA's solution is to introduce a "key-lock-commit" mechanism that can be implemented with four consecutive PB executions. This is to ensure all correct processes in a later view's broadcast phase can "convince" the left-out processes to proceed to the latest view after they lock themselves for a view change after "commit" [21]. Speeding Dumbo manages to reduce two invocations by introducing two additional steps of all-to-all communication in the agreement phase. They also introduce an effective "recovery" phase to allow processes that do not receive a message to retrieve a copy of this message from others using a "help me" function. A more recent work, HotStuff-M, introduces a design of PB with trusted hardware assistance. Processes in HotStuff-M receiving a valid message from the sender can attest to this message via a call to their trusted hardware in a specified log and sequence number [35].

Similar to HotStuff-M, we utilize trusted hardware for the attestation of received messages, and additionally, to facilitate the following agreement protocol. In our design, we keep a simple message exchange pattern (one-all-one-all) in the broadcast phase while ensuring that such reduction does not impact and can facilitate the progression of the agreement protocol. Our trusted PB has the following key features:

- **Propose**: The sender broadcasts a transaction with a monotonic counter number issued by the trusted hardware from the sender.
- **Endorse**: Processes return an endorsement message to the sender if the transaction is valid.
- **Cert**: The sender broadcasts a cert message if it has received $f+1$ endorsements from processes.

There are three states for a transaction: Void, Endorsed, and Certified. In general, a process records the state as Void if it is deemed invalid or if the transaction has not yet been delivered. It transitions to Endorsed once the transaction has been received and endorsed, and to Certified upon receiving a **Cert** message from the sender.

It is worth noting that all messages have to be authenticated inside the host's trusted hardware so that a process is unable to lie about the message it receives to arbitrarily update its recorded states. Further details and formal definitions are provided in Sec.IV and Sec.V, respectively.

²While different papers assign varied names and definitions, we adhere to a consistent set of reliable broadcast properties for easy comparison [37].

B. Agreement Phase

Recall the voting rules in the traditional BKR paradigm: a process votes 1 in an ABA instance only after the corresponding transaction is valid and delivered via RBC. A process votes 0 if the transaction fails to be delivered, and there are $n-f$ transactions that have been delivered. Similarly, the states for a transaction, determined by its delivery status in the PB instance, also play a crucial role in facilitating our agreement protocol. The state of a transaction determines the process's votes for that transaction. Since the states are stored inside the trusted hardware, a process cannot arbitrarily change the vote it intends to cast, as the states stored in the trusted hardware are immutable to arbitrary manipulation. The property weakness in the provable broadcast is compensated for in the agreement phase without incurring additional costs, thanks to the assistance of trusted hardware.

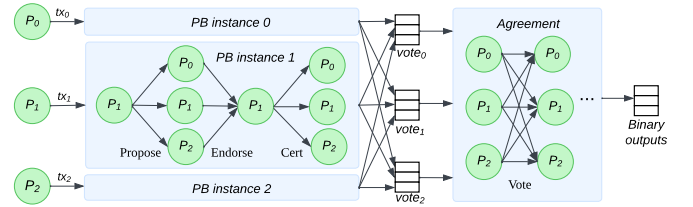


Fig. 1: Overall structure of JANUS

In contrast to the traditional ABA construction in ACS where each process casts a vote for every transaction, we merge the n vote messages into a single message with a n -sized vote vector, each element representing a vote for an individual Byzantine agreement for one of the n transactions. As shown in Fig 1, the vector input allows JANUS to merge the n parallel ABA instances into one agreement instance where a process exchanges its n -size vectors in a single message instead of sending n messages to n ABA instances. Specifically, a binary decision is independently achieved for each index ranging from 0 to $n-1$ of the vectors, corresponding to n ABA instances in the traditional BKR paradigm. As a result, the final output from our agreement protocol is a decision vector where each element represents the binary decision corresponding to a specific transaction. The binary decision for a transaction i is derived from the i -th element across all input vectors to the agreement protocol. As in Fig.2: $V_0, V_1, \dots, V_{n-1}$ are vote vectors from processes $P_0, P_1, \dots, P_{n-1}$, respectively.

Complexity reduction. The vector agreement in Janus requires all binary agreements for the n transactions to be resolved simultaneously, reducing both message and communication complexity. Compared to handling n separate vote messages in traditional ABA designs, Janus's vector agreement aggregates them into a single message with an n -size vector, reducing the number of authenticators from n to 1. In BFT protocols, the size of an authenticator typically ranges from 512 to 2048 bits, depending on the cryptographic algorithm, making it the dominant factor in a vote message's size. In traditional ABA designs, each process needs to broadcast n vote messages, leading to a total of $\mathcal{O}(n^2)$ authenticators

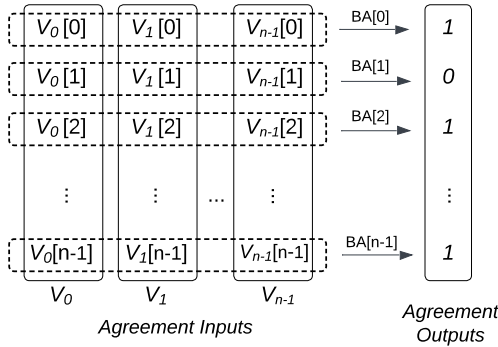


Fig. 2: Our agreement protocol with the vector inputs and outputs.

in one round of exchange. In contrast, Janus requires each process to broadcast only one vote message, resulting in $\mathcal{O}(n)$ authenticators per round, regardless of the network size n . This reduction in communication complexity becomes increasingly significant as the network size n grows.

A potential latency bottleneck. Unlike traditional ABA designs, where some binary instances may be completed earlier, Janus' vector agreement approach might raise concern that it could introduce latency concerns for Janus, potentially resulting in higher delays compared to traditional ABA in the BKR paradigm. However, we show that Janus' vector agreement protocol does not incur extra latency. Traditional parallel ABA instances also need to wait for the "slow ones" (the ones correct processes probably need to vote 0 for) to terminate for the ACS to finish. In other words, Janus requires processes to wait for $n-f$ deliveries to proceed to the agreement phase, while traditional ABA instances need to wait for $n-f$ of them to finish with positive outputs before voting 0 to the remaining slow ones. An ACS protocol's latency is measured based on the total time taken for the entire ACS protocol to complete and produce a final subset of transactions. In this case, Janus is even better than the traditional design, as the slow transactions in Janus can start after $n-f$ deliveries, whereas slow transactions in ABA can start only after $n-f$ agreements finish.

IV. LEVERAGING TRUSTED HARDWARE

In this section, we present how to configure and use trusted hardware components. Our approach leverages trusted components independent of specific hardware selections. Instead, our design focuses on the proof of concepts that incorporate general functionalities available to a wide range of trusted hardware options.

A. Trusted Components

Trusted components provide a physically isolated environment where data can be processed or stored safely (a.k.a., secure enclave). Data and internal processing within the enclave remain secure, even if the host is compromised, due to the trusted execution environment's immunity to arbitrary access.

JANUS employs two key trusted components: *counter* and *collector*. Each process is equipped with these two com-

ponents, both of which are accessible to the host via the configured *interfaces*. The *counter* assigns a counter number as a unique identifier for the message containing a new transaction that the host process intends to propagate. The *collector* keeps a record of the key internal states and facilitates the corresponding operations. This configuration enables processes to reject conflicting messages generated from equivocation, as such messages either lack valid identifiers or clash with the *collector*'s records. The *counter* increments monotonically every time a counter number is assigned, ensuring that each allocated number is distinct and unchangeable. The internal states in the *collector* can only be modified within the trusted hardware upon meeting certain conditions. Furthermore, access to the trusted components is restricted to the preconfigured interfaces, preventing any unauthorized access or operations.

Interfaces and messages. The trusted hardware provides several *interfaces* that enable the host process to invoke internal functions within the enclave. These interfaces are the exclusive means by which the host process can interact with its trusted components, and any invocation with invalid or irrelevant inputs will be rejected by the trusted components. Each process's trusted components offer five interfaces for message generation and internal state updates. The *counter* provides the following interface:

- **TEEpropose** is to issue a counter number for a transaction and return a **Propose** message to broadcast.

The *collector* component provides the following interfaces:

- **TEEendorse** is to validate the received **Propose** message and, if deemed valid, return a **Endorse** message to send back to the primary.
- **TEEcrt** is to validate $f+1$ **Endorse** messages for the same *transaction* and return a **Cert** message to broadcast.
- **TEEupdate** is to update internal states in secure storage.
- **TEEvote** is to return a **Vote** message, where the votes vector within this message corresponds to the internal states maintained in the secure storage.

The execution of these interfaces within the enclave is beyond the host's control, and the outcomes from these invocations remain tamper-proof, thanks to the enclave's integrated cryptographic module. In addition to the interfaces, the trusted components also have *internal functions* that can only be accessed and executed inside the secure enclave. We omit the detailed description of the *internal functions* here as their functions are simple and explicit in the construct pseudocodes Algorithm 1.

We configure our message types as follows:

- **Propose** (generated by the primary). The message contains the transaction to be broadcast. It is the only message whose generation invokes the *counter*.
- **Endorse** (by all processes). The message serves as a response to be sent back to the primary after a **Propose** message is received and verified.
- **Cert** (by the primary). The message is created after receiving $f+1$ valid **Endorse** messages.

- **Vote** (by all processes). The message contains an n -size vector that carries the vote values for the agreement instance.

B. Counter

Processes access and use counters only when intending to broadcast a transaction. The monotonic counter increments each time a primary process broadcasts a transaction, ensuring that the counter number assigned to transactions is unique and immutable. Since correct processes consistently increment the counter without omission and only issue a new counter number once the previous one is finalized, a process can expect transactions with a continuous sequence of counter numbers to be received as long as the corresponding primary is correct and participatory. As a result, processes are aware of the forthcoming expected counter numbers, and any message with an unexpected counter number cannot be handled as it is deemed invalid. Further details on handling messages with unexpected counter numbers refer to Sec.V-C.

C. Collector

The *collector* records and handles internal states crucial for the protocol's progression. Each process's *collector* records the current counter number and round number, along with an n -length list capturing the states of all n processes. Each element in the list represents the state of the current transaction from an individual process. Within process P_j 's secure storage, the i th element in the list records the current states of the transaction proposed by process P_i . Importantly, P_j 's vote on P_i 's transaction is determined by the states of the transaction.

The details of the usage of the internal states are given in Sec.V-B. The states for a transaction and their corresponding vote logic are listed as follows:

- Void: P_i 's **Propose** message has not yet been received. P_j will vote 0 in this case.
- Endorsed: P_j has received a valid **Propose** message and an **Endorse** message has been sent back to P_i . P_j will vote null in this case.
- Certified: P_j has received a valid **Cert** message. P_j will vote 1 in this case.
- Zeros: P_j has received n -f **Vote** message with 0. P_j will vote 0' in this case.
- Voted: P_j has generated a **Vote** message in this round. This is to prevent P_j from creating two **Vote** messages in the same round.

Updating internal states (Fig.3). The states within the trusted hardware (e.g., P_j for *instance_i*) get updated in response to the received messages. Importantly, a process can only update its internal states when the n -f input messages are valid, where messages (i) must be properly signed; (ii) come from the round that P_j is currently in; and (iii) come from n -f distinct processes. The updating rules are as follows:

- 0) The initial state is Void.
- 1) **Action-1:** Void $\rightarrow$ Endorsed.
Condition: P_j received a **Propose** message from P_i .
- 2) **Action-2:** Endorsed $\rightarrow$ Certified.

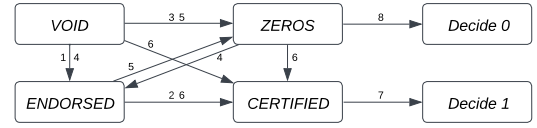


Fig. 3: Rules for updating states in trusted components

Condition: P_j received a **Cert** message from P_i .

- 3) **Action-3:** Void $\rightarrow$ Zeros.

Condition: P_j received n -f **Vote** messages where $v[i] = 0$, and the *CommonCoin* is 0.

- 4) **Action-4:** Void or Zeros $\rightarrow$ Endorsed.

Condition: P_j received a **Vote** message where $v[i] = \text{null}$ with the **Propose** message from P_i attached, and the *CommonCoin* is 1.

- 5) **Action-5:** Void or Endorsed $\rightarrow$ Zeros.

Condition: P_j received a **Vote** message where $v[i] = 0'$, and the *CommonCoin* is 0.

- 6) **Action-6:** Void or Endorsed or Zeros $\rightarrow$ Certified.

Condition: P_j received a **Vote** message where $v[i] = 1$.

- 7) **Decision-7:** Certified $\rightarrow$ 1 (accept).

Condition: P_j received n -f **Vote** messages where $v[i] = 1$, and the *CommonCoin* is 1.

- 8) **Decision-8:** Zeros $\rightarrow$ 0 (reject).

Condition: P_j received n -f **Vote** messages where $v[i] = 0'$, and the *CommonCoin* is 0.

D. Choice of the Trusted Hardware

While our trusted services are lightweight and generic enough to be potentially implemented in any trusted execution environment, we use SGX as its Linux SDK provides a convenient development environment. For instance, JANUS can be implemented with AMD SEV. The two trusted services of JANUS, counter and collector, can be realized in AMD with its AMD SEV's secure nested paging (attention function) and encrypted memory.

E. Scalability and Practicality of Using SGX in JANUS

The trusted memory in JANUS consists of an n -size list along with two numbers (current counter number and round number). The list records the internal states of n transactions. Given that there are 4 possible states, the overall size of the trusted memory is approximately $3n+64$ bits. In a network of 300 processes, for instance, the trusted memory required for the JANUS operation is less than 1 KB, which is an acceptable size for most of the commercially available trusted hardware. It's worth noting that in JANUS' implementation, we avoid using packages involving thread function and all the simulation results are obtained with a single thread processing. Multithread processing is not required in JANUS as the multithreading cannot improve the performance of JANUS to a large extent. The parallelization in JANUS is more of high-level message processing. Compared with previous work like Hybster with multiple TEE instances for parallel processing, JANUS only requires one TEE instance no matter how many transactions are being handled concurrently.

F. Potential Limitations and Room for Improvement

Similar to all other existing hybrid BFT systems with optimal resilience (tolerating malicious minority), the assumed security of the trusted hardware component may not be easily guaranteed in the currently available off-the-shelf products. However, regardless of this limitation, hybrid protocols leveraging trusted hardware have been adopted in practice (e.g. MinBFT in Hyperledger Fabric, and SGX in Flashbots to build Ethereum blocks).

V. JANUS CONSTRUCTION

In this section, we present the full JANUS protocol given as pseudocode in Algorithm 1.^{3,4}

A. Trusted Provable Broadcast Protocol

Our trusted PB protocol is structured into three phases (see Fig.1): *propose*, *endorse*, and *certify*. Each phase aligns with the type of message broadcasted during its respective phase. Here, we outline the abstracted workflow.

Step-①: Propose. At each process P_i , P_i generates a **Propose** message with a unique counter number and broadcasts it to others (line 15). Upon receiving the **Propose** message from P_i , other processes verify the signature and counter number within their trusted components (line 21). If valid, they generate an **Endorse** message and send it back to P_i (line 22). The states of P_i within the secure storage of those processes are then updated accordingly: the status is set to Endorsed (line 57).

Step-②: Endorse. Upon receiving $f+1$ **Endorse** messages, P_i verifies the validity of these messages in trusted components and subsequently creates a **Cert** message to broadcast (line 17).

Step-③: Certify. Upon receiving a valid **Cert** message from P_i , other processes update P_i 's status to Certified (line 24).

B. Agreement Protocol with Vector Inputs

The consensus algorithm aims to guarantee that correct processes share the same set of delivered transactions for inclusion in ACS. We build a vector of votes within secure enclaves from the internal states and then encapsulate it into a **Vote** message. We outline three distinct steps.

Step-④: Generating a Vote message. At each process P_i , P_i engages in all n transaction broadcast instances, and the delivery statuses of these transactions are recorded in P_i 's secure storage. Among all n broadcast instances, at least $n-f$ of them guarantee successful delivery eventually as they are led by correct processes. When P_i has at least $n-f$ Certified statuses in its secure storage in round 1 (the first round), it will call its trusted interface to produce a **Vote** message to broadcast (line 95). Upon invocation, the interface returns an

n -sized vector that contains P_i 's votes for the n transactions. Votes are determined by the status of the transactions in P_i 's secure storage and the mapping rules are as given in Sec.IV-C.

While we eventually obtain binary outcomes for all transactions in the agreement phase, the votes themselves are not strictly binary as they go beyond just 0 and 1. In other words, the input vector to the agreement protocol contains non-binary values, while the output vector (consensus outcome) only contains 0 and 1. Typically, a vote of 1 represents acceptance of a transaction, while a vote of 0 means rejection. However, in our approach, P_i votes null when the transaction is valid and endorsed by P_i , but no subsequent **Cert** message is received from the primary (line 103). This indicates that P_i recognizes the transaction's validity but is unsure whether the proof has been generated.

Step-⑤: Exchanging and updating the Vote message. After broadcasting the generated **Vote** message, P_i awaits the reception of $n-f$ **Vote** messages and adjusts the vote messages accordingly (line 29). Similar to the reproposable ABA [16] with a *bias towards 1*, the updating logic is designed to give preference to 1, increasing its likelihood of becoming the final decision. If P_i receives a valid **Vote** message proposing 1 for an instance (an element of the vector is 1) and P_i 's own vote is 0 or null, P_i will change its vote to 1 (line 81).

Step-⑥: Flipping the coin and converging to agreements. After collecting $n-f$ **Vote** messages, a common coin is tossed, and P_i endeavors to achieve an agreement when conditions are met (line 30). Notably, the common coin value is optimistically set to 1 in the first round (round 1) before outputting either 0 or 1 with equal probability in subsequent rounds.

For each index x , ranging from 0 to $n-1$, in an n -size vector representing n processes (line 98):

- Upon receiving $n-f$ **Vote** messages where elements x are all 1, P_i decides 1 for x if the coin value is 1.
- Upon receiving $n-f$ **Vote** messages where elements x are all 0, P_i votes 0' for x if the coin value is 0.
- Upon receiving $n-f$ **Vote** messages where elements x are all 0', P_i decides 0 for x if the coin value is 0.

C. Special cases

Blockage situation. The distinction in deciding conditions between 1 and 0 (including the involvement of vote 0') is designed to prevent a deadlock scenario that could compromise the *termination* property, potentially stalling the protocol indefinitely and thereby breaking the system's *liveness*.

A blockage situation might happen when votes from correct processes are divided between null and 0. It happens when some correct processes have received the **Propose** message while others have not. However, none of them have received the **Cert** message to be converted to vote 1. Consequently, the correct processes might become stuck and fail to reach a consensus because none of the decisive conditions for consensus can be met as long as the Byzantine processes choose not to send any message.

In this case, we assert that the relevant transaction is valid, as a null vote can be produced only when the corresponding

³We exclude the status **Voted** in pseudocode for simplicity. Each process's internal status within the trusted hardware is temporarily set to **Voted** once a **Vote** message is successfully generated. Before the creation of a new **Vote** message in the subsequent round, this status remains unchanged. This prevents faulty processes from creating another **Vote** message within the same round using a different set of received messages.

⁴We exclude the well-known [38] procedures of termination and message processing in the pseudocode for simplicity.

Algorithm 1 JANUS Construction

```

1: Parameters:
2:    $n$ : total number of processes.
3:    $f < n/2$ : maximum number of Byzantine processes.
4: Variables:
5:    $T$ : the transaction a primary intends to propose
6:    $p$ : primary's ID, this is also its instance's ID
7:    $k$ : counter number
8:    $\sigma$ : the signature for message authentication
9:    $coin$ : the binary common coin value
10:   $v[]$ : vote values (vector)
11:   $r$ : the round number, starting from 1
12: As a primary  $p$ 
13: upon event  $p$  proposes transaction  $T$  do
14:    $\langle \text{Propose}, T, k, \sigma_p \rangle \leftarrow \text{TEEpropose}(T)$ 
15:   Broadcast  $\langle \text{Propose}, T, k, \sigma_p \rangle$ 
16: upon event receiving Endorse messages from  $f+1$  processes do
17:    $\langle \text{Cert}, h, k, \sigma_p \rangle \leftarrow \text{TEEcet}(f+1 \text{ Endorse messages})$ 
18:   Broadcast  $\langle \text{Cert}, h, k, \sigma_p \rangle$ 
19: Each process  $i$ 
20: upon event receiving a Propose message from  $p$  do
21:    $\langle \text{Endorse}, h, k, \sigma_i \rangle \leftarrow \text{TEEendorse}(\langle \text{Propose}, T, k, \sigma_p \rangle)$ 
22:   Send  $\langle \text{Endorse}, h, k, \sigma_i \rangle$  to  $p$ 
23: upon event receiving a Cert message from  $p$  do
24:    $\text{TEEupdate}(\langle \text{Cert}, h, k, \sigma_p \rangle)$ 
25: upon event receiving  $n-f$  Vote messages do
26:   //for the first round, the coin value is 1
27:    $coin \leftarrow \text{CommonCoin}(n-f \sigma, r)$ 
28:   for  $\langle \text{Vote}, v_j, k, r, \sigma_j \rangle$  in  $n-f$  received messages
29:    $\text{TEEupdate}(\langle \text{Vote}, v_j, k, r, \sigma_j \rangle)$ 
30:   for index  $x$  in  $[0..n-1]$ 
31:     //try to reach an agreement for each instance
32:     if  $\exists f+1$  Vote messages where  $v[x] == 1$  and  $coin == 1$  then
33:       Decide 1 for  $instance_x$ 
34:     if  $\exists f+1$  Vote messages where  $v[x] == 0$  and  $coin == 0$  then
35:       Decide 0 for  $instance_x$ 
36:   if all instances have been decided
37:     //indexes and hash values of included transactions
38:     //components in an ACS message are retrieved from secure
     storage
39:     Broadcast  $\langle \text{ACS}, k, index[], hash[], \sigma_i \rangle$ 
40:   else
41:     //rebroadcast the Vote message for next round
42:      $\langle \text{Vote}, v, k, r, \sigma_i \rangle \leftarrow \text{TEEvote}()$ 
43:     Broadcast  $\langle \text{Vote}, v, k, r, \sigma_i \rangle$ 
44: Trusted components
45: interface  $\text{TEEpropose}(T)$ :
46:    $k \leftarrow \text{counter.issue}()$  //internal function: return a new counter number
47:    $\text{counter.increment}()$ 
48:    $\sigma \leftarrow \text{sign}(\text{Propose}, T, k)$  //same procedure to authenticate other
     messages
49:   return  $\langle \text{Propose}, T, k, \sigma \rangle$ 
50: interface  $\text{TEEendorse}(\langle \text{Propose}, T, k, \sigma \rangle)$ :
51:   if  $k \neq \text{collector.counterNo}()$  //verify the counter number
52:     break
53:   if  $\text{verify}(\sigma) == \text{false}$  //verify the signature
54:     break
55:    $h \leftarrow \text{hash}(\langle \text{Propose}, T, k, \sigma \rangle)$ 
56:   //the states for  $p$  is updated from the default Void
57:    $\text{collector}(p) \leftarrow \text{Endorsed}$ 
58:   return  $\langle \text{Endorse}, h, k, \sigma \rangle$ 
59: interface  $\text{TEEcet}(f+1 \text{ Endorse messages})$ :
60:   //check if there is any message with invalid signature
61:   if  $\exists \langle \text{Endorse}, h, k, \sigma_j \rangle$  where  $\text{verify}(\sigma_j) == \text{false}$ 
62:     break
63:   //check if any message doesn't endorse the correct Propose message
64:   if  $\exists \langle \text{Endorse}, h, k, \sigma_j \rangle$  where  $h \neq \text{hash}(\langle \text{Propose}, T, k, \sigma \rangle)$ 
65:     break
66:   //check if there is any message with incorrect counter number
67:   if  $\exists \langle \text{Endorse}, h, k, \sigma_j \rangle$  where  $k \neq \text{collector.counterNo}()$ 
68:     break
69:   //check if there is any duplicate messages
70:   if  $\exists \langle \text{Endorse}, h, k, \sigma_j \rangle$  and  $\langle \text{Endorse}, h, k, \sigma_m \rangle$  where  $\sigma_j == \sigma_m$ 
71:     break
72:    $h \leftarrow \text{hash}(\langle \text{Propose}, T, k, \sigma \rangle)$ 
73:    $k \leftarrow \text{collector.counterNo}()$  //the current recorded counter number
74:   return  $\langle \text{Cert}, h, k, \sigma \rangle$ 
75: interface  $\text{TEEupdate}(\text{input})$ : //updating the internal states
76:   //all input message(s) are checked for signature validity
77:   //all input message(s) are checked for counter no. and round no.
     correctness
78:   //verification processes are similar to TEEcet, omitted for simplicity
79:   if the input is  $\langle \text{Cert}, h, k, \sigma_p \rangle$ 
80:      $\text{collector}(p) \leftarrow \text{Certified}$ 
81:   if the input is  $\langle \text{Vote}, v, k, r, \sigma_p \rangle$  where  $v[p] == 1$ 
82:      $\text{collector}(p) \leftarrow \text{Certified}$ 
83:   //the inputs here consist of a sequential input of individual messages
84:   if the inputs are  $n-f \langle \text{Vote}, v, k, r, \sigma_p \rangle$  where  $v[p] == 0$  and  $coin$ 
      $== 0$ 
85:      $\text{collector}(p) \leftarrow \text{Zeros}$ 
86:   if the input is  $\langle \text{Vote}, v, k, r, \sigma_p \rangle$  where  $v[p] == 0$  and  $coin == 0$ 
87:      $\text{collector}(p) \leftarrow \text{Zeros}$ 
88:   if the input is  $\langle \text{Vote}, v, k, r, \sigma_p \rangle$  w/  $\langle \text{Propose}, T, k, \sigma_p \rangle$  attached
     where  $v[p] == \text{null}$  and  $coin == 1$ 
89:      $\text{collector}(p) \leftarrow \text{Endorsed}$ 
90:   //a partitioned process catches up
91:   if the input is  $\langle \text{ACS}, k, index[], hash[], \sigma_i \rangle$  where  $k > \text{collector.counterNo}()$ 
92:      $\text{counter} \leftarrow k$  //its counter number skips to  $k$ 
93:      $\text{collector}(n \text{ processes}) \leftarrow \text{Void}$  //its states for all processes
     update to Void
94: upon event  $i$  has at least  $n-f$  Certified states in its secure storage in
     round 1 do
95:    $\langle \text{Vote}, v, k, I, \sigma_p \rangle \leftarrow \text{TEEvote}()$ 
96:   Broadcast  $\langle \text{Vote}, v, k, I, \sigma_p \rangle$ 
97: interface  $\text{TEEvote}()$ : //mapping the internal states to corresponding votes
98:   for  $\text{collector.state}(x)$  in  $\text{collector.state}(0..n-1)$ 
99:     if  $\text{collector.state}(x) == \text{Certified}$ 
100:        $v[x] \leftarrow 1$ 
101:     if  $\text{collector.state}(x) == \text{Void}$ 
102:        $v[x] \leftarrow 0$ 
103:     if  $\text{collector.state}(x) == \text{Endorsed}$ 
104:        $v[x] \leftarrow \text{null}$ 
105:     if  $\text{collector.state}(x) == \text{Zeros}$ 
106:        $v[x] \leftarrow 0$ 
107:    $\text{collector.increment}(r)$  //increment the round number
108:    $k \leftarrow \text{collector.counterNo}()$ 
109:   return  $\langle \text{Vote}, k, v, r, \sigma \rangle$ 

```

Propose message is authenticated within the trusted components. As a result, both 1 and 0 can be acceptable as long as correct processes eventually converge to the same decision. The general logic to solve the blockage is: (i) a null vote can convert 0 to null and eventually decide 1 if the common coin is 1; (ii) a 0 vote can convert null to 0 and eventually decide 0 if the common coin is 0. Specifically, we solve this issue by enabling the following conditions if there is no decision made after round 5: (i) a null vote will have the original **Propose** message attached for the next round's broadcast if the common coin is 1; (ii) a 0 vote is converted to null upon receiving a null vote with a valid **Propose** message attached if the common coin is 1; (iii) a 1 vote is created upon receiving $f+1$ null votes if the common coin is 1; (iv) a 0' vote is created upon receiving $f+1$ 0 or null votes if the common coin is 0; (v) a 0' vote is created upon receiving $f+1$ 0 or null votes if the common coin is 0; (vi) a null vote is converted to 0 upon receiving a 0' vote if the common coin is 0.

In summary, the agreement procedure tends toward deciding 1 when the common coin is 1 or toward deciding 0 when the common coin is 0. The null votes and 0 votes interchangeably "flip" until the common coin facilitates one of the two decisive conditions, leading to a universal decision.

Network partition. Under the assumption of asynchrony, challenges may arise where a correct process experiences delays in receiving messages from other processes due to network latency, causing it to lag behind in progress. To mitigate this issue, we implement an arranging window mechanism, similar to that used in protocols like Hybster [30]. This window is defined by the counter number of the last completed ACS, serving as the lower watermark, with the upper bound set slightly higher. Only transactions within this range are considered valid. Once an ACS is achieved, correct processes broadcast **ACS** messages. These messages include the counter number (as transactions within an ACS share the same counter number), along with the indexes and hash values of included transactions. A lagging process can catch up by collecting a quorum of **ACS** messages and retrieving all transactions from finalized ACSs from other processes upon request.

In an asynchronous network, messages might be delivered in a different order than their initial orders. Hence, if a process receives a new message with a higher counter number from a primary while it is working on a lower-numbered transaction, it temporarily stores the new message outside trusted components until preceding transactions from the same primary are finalized. Importantly, a process will only store transactions with counter numbers within the ordering window. Transactions with counter numbers higher than the high watermark will be discarded, as correct processes will issue any counter number before the previous one is finalized and their transactions can hardly have far-advanced counter numbers. This prevents a faulty process from overwhelming the external storage of correct processes by sending multiple messages with far subsequent counter numbers. Nonetheless, the counter number in an **ACS** message may exceed the ordering window

due to network partitions, risking the rejection of the latest transactions by lagging processes. In this case, an up-to-date **ACS** message can synchronize the slow process and update its ordering window.

VI. ANALYSIS AND EVALUATION

A. Complexity Analysis

We analyze both the *message complexity*, indicating the total number of messages exchanged among processes, as well as the *communication complexity*, which is the total number of bits exchanged. JANUS' message complexity is $\mathcal{O}(n^2)$ and $\mathcal{O}(n)$ per transaction while the overall communication complexity is asymptotically $\mathcal{O}(3n^2 + |m|n^2 \log n)$. The $3n^2$ is due to n parallel instances of provable broadcast with three communication steps and $|m|n^2 \log n$ comes from the agreement phase where $|m|$ is the **Vote** message size and $\log n$ indicates the number of **Vote** message exchange rounds required for reaching agreement. A **Vote** message includes a $2n$ -bit vote vector along with metadata such as process identifiers and signatures, which predominate over the vote vector in size when n is within a reasonable range.

B. Security Analysis

We prove that the JANUS protocol solves ACS in a system with at least $n \geq 2f + 1$ processes, where f represents the number of Byzantine processes.

Theorem 1 (JANUS BA). *A BA instance x in the JANUS protocol achieves BA-agreement, BA-termination, BA-validity.*

Theorem 2 (JANUS ACS). *JANUS guarantees ACS-validity, ACS-agreement, and ACS-totality.*

The full proofs are deferred to Appendix B.

C. Evaluation

We benchmark JANUS against HoneybadgerBFT to assess the performance of an asynchronous BFT protocol, and against MinBFT and Damysus for comparison within the context of trusted protocols [23][39][24]. Other ACS protocols, such as PACE [16], were not considered due to the unavailability of code. We give our evaluations below.

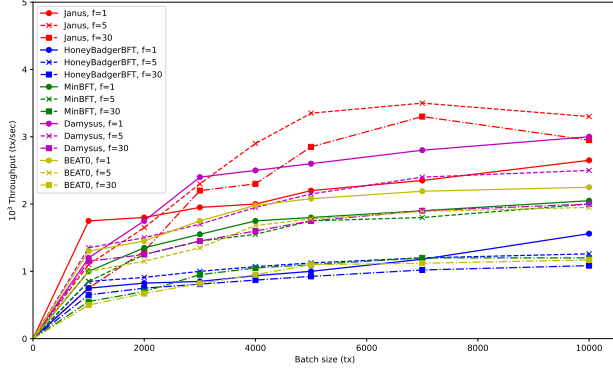
Configuration. Building on the open-source ResilientDB framework [40], we implement our protocol in Python and deploy it on the Google Cloud platform, leveraging its built-in Intel SGX capabilities. We implement JANUS in a geo-distributed network configuration comprising *E2-standard-8* machines with 8 vCPUs and 32GB memory, deployed across several regions: Sydney, Tokyo, and Atlanta. The measured experimental settings of the network are presented in Table I. The transaction size is set to 250 bytes, and each data point represents the average results over 20 runs.

As for the trusted enclave, we use Intel SGX's Linux SDK to implement the abstraction of the trusted components at each process [41]. While we acknowledge the potential vulnerabilities of SGX [42][28][43], we use it only to demonstrate the performance due to its convenient development

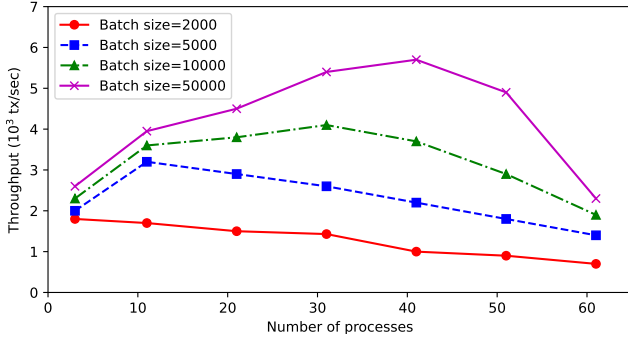
TABLE I: Round trip time & bandwidth between each region

	Sydney-Tokyo	Tokyo-Atlanta	Sydney-Atlanta
Round trip time (ms)	179	254	113
Bandwidth (Mbps)	53.4	132.5	65.0

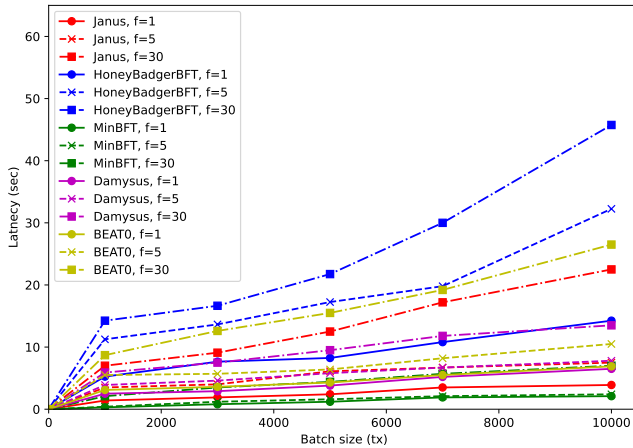
environment as SGX still remains a widely adopted platform for implementation (e.g. [24][30]).



(a) Throughput for different batch sizes



(b) Throughput with varying numbers of processes



(c) Latency for different batch sizes

Fig. 4: Performance Evaluation

Throughput analysis (see Fig.4a). The JANUS protocol

demonstrates enhanced performance over trusted protocols like Damysus (chained version) and MinBFT. This advantage becomes more evident with larger batch sizes due to JANUS' parallel design for concurrent processing. Compared to MinBFT, JANUS shows an 89% throughput improvement on average. Particularly, JANUS outperforms minBFT in relatively larger networks due to the additional benefits of parallelisation JANUS offers. It's notable that in smaller networks with fewer than 10 processes, Damysus slightly outperforms JANUS for larger batches. This is attributed to Damysus' streamlined design with pipeline design allowing it to run concurrently (with limited parallelisation), while JANUS, being an ACS protocol optimized for parallel invocation, may not scale as effectively with fewer processes. As shown in the figure above, BEAT performs well in small networks. However, in relatively larger networks, Janus outperforms BEAT. This is due to the improved message complexity in Janus.

In comparison with HoneyBadgerBFT, JANUS achieves significantly higher throughput rates, due to the reduced communication and message complexity. With an increased batch size, the disparity in performance becomes even more evident. While HoneyBadgerBFT's throughput plateaus, the throughput of JANUS continues to scale, reflecting a growth pattern that suggests superior management of larger transaction loads. Particularly in the range of 8,000 to 10,000 transactions, JANUS maintains a consistent upward trajectory, highlighting its robustness in high-load scenarios.

The throughput of JANUS slightly drops when the batch size goes beyond 7,000. This might be the case when the workload exceeds capacity. In addition, increasing the number of processes may elevate the probability of requiring additional rounds to complete the agreement phase (i.e., exchanging the vote vectors). This introduces a trade-off scenario, where the advantages of parallelism clash with the growing volume of messages to be transmitted and processed as the number of processes escalates.

To further explore this trade-off, we carry out experiments to evaluate the performance of JANUS with varying numbers of processes for different batch sizes. We demonstrate that throughput increases with an increased number of processes up to a peak point before it begins to decline. The peak point is influenced by the batch sizes as shown in Fig.4b.

Latency analysis. As shown in Fig.4c, JANUS maintains an overall lower latency than HoneyBadgerBFT, with an improvement margin that becomes more significant as the network scales. The improvement mostly stems from reduced message propagation required to achieve the same level of throughput. MinBFT and Damysus present better latency performance, especially in larger networks. This is expected as BFT protocols designed for partially synchronous networks are known to require fewer communication rounds (i.e., MinBFT) and/or communication complexity (i.e., Damysus). Compared with BEAT, Janus' latency is approximately 21%-37% lower across networks of various batch sizes. Janus achieves better latency performance because it requires fewer total processes

to tolerate the same number of faults.

To provide a more detailed understanding, we provide a breakdown latency analysis of JANUS and HoneyBadgerBFT according to the two phases of BKR paradigm (cf. Fig.5). To ensure a fair comparison, we keep the total number of processes n the same for both protocols, even though JANUS can tolerate more Byzantine processes for the same n . The batch size is set to 25,000 to evaluate performance under heavy loads. As shown in Fig.5, latencies in both phases of JANUS are significantly lower than those of HoneyBadgerBFT, as expected. While the agreement phase remains a large portion of the overall running time for both protocols, the agreement phase in JANUS is much faster than that of HoneyBadgerBFT due to the message aggregation in JANUS' agreement protocol. Instead of waiting for $n-f$ ABA instances to produce a positive outcome before initiating the "slow" instances, the aggregated vector in JANUS' agreement protocol allows all n binary agreements to commence simultaneously, without the need to spend additional time to trigger the start on the slow ones.

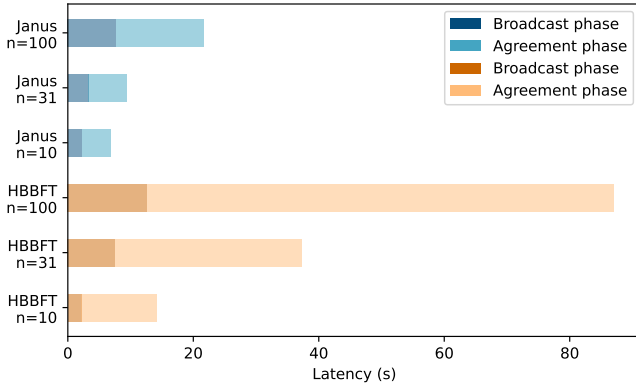


Fig. 5: Latency breakdown analysis between JANUS and HoneyBadgerBFT (HBBFT)

VII. CONCLUSION

In this paper, we propose JANUS, a new hybrid Asynchronous Common Subse (ACS) protocol, by leveraging trusted components to reduce *message complexity* from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$ and achieve optimal *resilience* from $3f+1$ to $2f+1$. Our protocol lies in two innovated foundations: (i) replacing the reliable broadcast with a trusted provable broadcast (supported by trusted components) and (ii) developing a resilient binary agreement protocol capable of aggregating concurrent instances into a vector (termed as *aggregated vectors*).

We implement and evaluate JANUS, with experimental results showing performance improvements over both ACS constructions (1.92X, HoneyBadgerBFT/BEAT) and hybrid protocols tolerating Byzantine minorities under partially synchronous networks (1.49X, MinBFT/Damysus).

REFERENCES

- [1] S. Duan and H. Zhang, "Recent progress on BFT in the era of blockchains," *National Science Review*, vol. 9, no. 10, p. nwac132, 2022.
- [2] A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song, "The honey badger of BFT protocols," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 31–42.
- [3] S. Duan, M. K. Reiter, and H. Zhang, "BEAT: Asynchronous BFT made practical," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 2028–2041.
- [4] C. Liu, S. Duan, and H. Zhang, "EPIC: Efficient asynchronous BFT with adaptive security," in *IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2020, pp. 437–451.
- [5] M. Ben-Or, "Another advantage of free choice (extended abstract) completely asynchronous agreement protocols," in *Proceedings of the ACM symposium on Principles of Distributed Computing (PODC)*, 1983, pp. 27–30.
- [6] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and efficient asynchronous broadcast protocols," in *Annual International Cryptology Conference (CRYPTO)*. Springer, 2001, pp. 524–541.
- [7] K. Kursawe and V. Shoup, "Optimistic asynchronous atomic broadcast," in *International Colloquium on Automata, Languages, and Programming (ICALP)*. Springer, 2005, pp. 204–215.
- [8] H. V. Ramasamy and C. Cachin, "Parsimonious asynchronous byzantine-fault-tolerant atomic broadcast," in *International Conference On Principles Of Distributed Systems (PODC)*. Springer, 2005, pp. 88–102.
- [9] E. Kokoris Kogias, D. Malkhi, and A. Spiegelman, "Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020, pp. 1751–1767.
- [10] S. Das, T. Yurek, Z. Xiang, A. Miller, L. Kokoris-Kogias, and L. Ren, "Practical asynchronous distributed key generation," in *IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 2518–2534.
- [11] S. Das, Z. Xiang, L. Kokoris-Kogias, and L. Ren, "Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling," in *USENIX Security Symposium (USENIX Security)*, 2023, pp. 5359–5376.
- [12] A. Chopard, M. Hirt, and C.-D. Liu-Zhang, "On communication-efficient asynchronous MPC with adaptive security," in *International Conference on Theory of Cryptography (TCC)*. Springer, 2021, pp. 35–65.
- [13] D. Lu, T. Yurek, S. Kulshreshtha, R. Govind, A. Kate, and A. Miller, "Honeybadgermpc and asynchomix: Practical asynchronous mpc and its application to anonymous communication," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2019, pp. 887–903.
- [14] S. Duan, X. Wang, and H. Zhang, "FIN: Practical signature-free asynchronous common subset in constant time," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.
- [15] M. Ben-Or, B. Kelmer, and T. Rabin, "Asynchronous secure computations with optimal resilience," in *ACM symposium on Principles Of Distributed Computing (PODC)*, 1994, pp. 183–192.
- [16] H. Zhang and S. Duan, "PACE: Fully parallelizable BFT from reposable Byzantine agreement," *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022.
- [17] Z. Zhang, Y. Zhou, S. Duan, H. Zhang, B. Hu, L. Wang, and J. Liu, "Dory: Faster asynchronous BFT with reduced communication for permissioned blockchains," *Cryptology ePrint Archive*, 2022.
- [18] Y. Lu, Z. Lu, Q. Tang, and G. Wang, "Dumbo-MVBA: Optimal multi-valued validated asynchronous Byzantine agreement, revisited," in *Proceedings of the Symposium on Principles of Distributed Computing (PODC)*, 2020, pp. 129–138.
- [19] B. Guo, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, "Dumbo: Faster asynchronous BFT protocols," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2020, pp. 803–818.
- [20] Y. Gao, Y. Lu, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, "Dumbo-NG: Fast asynchronous BFT consensus with throughput-oblivious latency," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 1187–1201.
- [21] I. Abraham, D. Malkhi, and A. Spiegelman, "Asymptotically optimal validated asynchronous byzantine agreement," in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 337–346.
- [22] B. Guo, Y. Lu, Z. Lu, Q. Tang, J. Xu, and Z. Zhang, "Speeding Dumbo: Pushing asynchronous BFT closer to practice," *Network and Distributed System Security (NDSS)*, 2022.

- [23] G. S. Veronese, M. Correia, A. N. Bessani, L. C. Lung, and P. Verissimo, "Efficient Byzantine fault-tolerance," *IEEE Transactions on Computers (TC)*, vol. 62, no. 1, pp. 16–30, 2011.
- [24] J. Decouchant, D. Kozhaya, V. Rahli, and J. Yu, "DAMYSUS: streamlined BFT consensus leveraging trusted components," in *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2022, pp. 1–16.
- [25] C. Stathakopoulou, S. Rüsch, M. Brandenburger, and M. Vukolić, "Adding fairness to order: Preventing front-running attacks in bft protocols using TEEs," in *International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 2021, pp. 34–45.
- [26] G. Zhang and H.-A. Jacobsen, "Prosecutor: An efficient BFT consensus algorithm with behavior-aware penalization against byzantine attacks," in *Proceedings of the International Middleware Conference (Middleware)*, 2021, pp. 52–63.
- [27] P. Verissimo, "Traveling through wormholes: Meeting the grand challenge of distributed systems," in *Proc. Int. Workshop on Future Directions in Distributed Computing (FuDiCo)*, 2002, pp. 144–151.
- [28] R. Li, Q. Wang, Q. Wang, D. Galindo, and M. Ryan, "SoK: TEE-assisted confidential smart contract," *Proceedings on Privacy Enhancing Technologies (PETS)*, vol. 3, pp. 711–731, 2022.
- [29] J. Decouchant, D. Kozhaya, V. Rahli, and J. Yu, "OneShot: View-adapting streamlined BFT protocols with trusted execution environments," in *International Parallel and Distributed Processing Symposium (IPDPS)*, 2024.
- [30] J. Behl, T. Distler, and R. Kapitza, "Hybrids on steroids: SGX-based high-performance BFT," in *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2017, pp. 222–237.
- [31] D. Levin, J. R. Douceur, J. R. Lorch, and T. Moscibroda, "TrInc: Small trusted hardware for large distributed systems," in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, vol. 9, 2009, pp. 1–14.
- [32] S. Gupta, S. Rahnama, S. Pandey, N. Crooks, and M. Sadoghi, "Dissecting BFT consensus: In trusted components we trust!" in *European Conference on Computer Systems (EuroSys)*, 2023, pp. 521–539.
- [33] B.-G. Chun, P. Maniatis, S. Shenker, and J. Kubiatowicz, "Attested append-only memory: Making adversaries stick to their word," *ACM SIGOPS Operating Systems Review (OSR)*, vol. 41, no. 6, pp. 189–204, 2007.
- [34] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "HotStuff: BFT consensus with linearity and responsiveness," in *ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 347–356.
- [35] S. Yandamuri, I. Abraham, K. Nayak, and M. K. Reiter, "Communication-efficient BFT using small trusted hardware to tolerate minority corruption," in *International Conference on Principles of Distributed Systems (OPODIS)*. Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2023.
- [36] A. Bessani, M. Correia, T. Distler, R. Kapitza, P. Esteves-Verissimo, and J. Yu, "Vivisection the dissection: On the role of trusted components in BFT protocols," *arXiv preprint arXiv:2312.05714*, 2023.
- [37] C. Cachin, R. Guerraoui, and L. Rodrigues, "Introduction to reliable and secure distributed programming," in *Springer Science & Business Media*, 2011.
- [38] A. Mostéfaoui, H. Moumen, and M. Raynal, "Signature-free asynchronous Byzantine consensus with $t < n/3$ and $o(n^2)$ messages," in *ACM Symposium on Principles of Distributed Computing (PODC)*, 2014, pp. 2–9.
- [39] B. Sylvain, M. Andrew, dantengsky, and S. Brian, "Honey-BadgerBFT Github," in *Retrieved in https://github.com/initc3/HoneyBadgerBFT-Python/*, 2023.
- [40] cjcchen *et al.*, "ResilientDB Github," in *Retrieved in https://github.com/resilientdb/resilientdb/tree/resilientdb-legacy-eurosys23*, 2023.
- [41] V. Costan and S. Devadas, "Intel SGX explained," *Cryptology ePrint Archive*, 2016.
- [42] K. Murdock, D. Oswald, F. D. Garcia, J. Van Bulck, D. Gruss, and F. Piessens, "Plundervolt: Software-based fault injection attacks against Intel SGX," in *IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1466–1482.
- [43] S. Van Schaik, A. Kwong, D. Genkin, and Y. Yarom, "SGAxe: How SGX fails in practice," *https://sgaxe.com/files/SGAxe.pdf*, 2020.
- [44] M. Vukolić, "Quorum systems: With applications to storage and consensus," vol. 9, 2012.
- [45] X. Wang, S. Duan, J. Clavin, and H. Zhang, "BFT in blockchains: From protocols to use cases," *ACM Computing Surveys (CSUR)*, vol. 54, no. 10s, pp. 1–37, 2022.
- [46] N. Ben-David, B. Y. Chan, and E. Shi, "Revisiting the power of non-equivocation in distributed protocols," in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 2022.
- [47] A. Clement, F. Junqueira, A. Kate, and R. Rodrigues, "On the (limited) power of non-equivocation," in *ACM symposium on Principles of Distributed Computing (PODC)*, 2012, pp. 301–308.
- [48] M. Castro and B. Liskov, "Practical Byzantine fault tolerance and proactive recovery," *ACM Transactions on Computer Systems (TOCS)*, vol. 20, no. 4, pp. 398–461, 2002.
- [49] P. Sousa, N. F. Neves, and P. Verissimo, "How resilient are distributed fault/intrusion-tolerant systems?" in *International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2005, pp. 98–107.
- [50] M. Correia, N. F. Neves, and P. Verissimo, "How to tolerate half less one byzantine nodes in practical distributed systems," in *IEEE International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 2004, pp. 174–183.
- [51] G. S. Veronese, M. Correia, A. N. Bessani, and L. C. Lung, "EBAWA: Efficient byzantine agreement for wide-area networks," in *IEEE International Symposium on High Assurance Systems Engineering (HASE)*. IEEE, 2010, pp. 10–19.
- [52] R. Kapitza, J. Behl, C. Cachin, T. Distler, S. Kuhnle, S. V. Mohammadi, W. Schröder-Preikschat, and K. Stengel, "CheapBFT: Resource-efficient Byzantine fault tolerance," in *ACM European Conference on Computer Systems (EuroSys)*, 2012, pp. 295–308.
- [53] J. Liu, W. Li, G. O. Karame, and N. Asokan, "Scalable Byzantine consensus via hardware-assisted secret sharing," *IEEE Transactions on Computers (TC)*, vol. 68, no. 1, pp. 139–151, 2018.
- [54] M. Ben-Or, R. Canetti, and O. Goldreich, "Asynchronous secure computation," in *ACM Symposium on Theory of Computing (STOC)*, 1993, pp. 52–61.
- [55] H. Zhang, S. Duan, B. Zhao, and L. Zhu, "WaterBear: Practical asynchronous BFT matching security guarantees of partially synchronous BFT," in *USENIX Security Symposium (USENIX Security)*, 2023, pp. 5341–5357.
- [56] N. F. Neves, M. Correia, and P. Verissimo, "Solving vector consensus with a wormhole," *IEEE Transactions on Parallel and Distributed Systems (TPDS)*, vol. 16, no. 12, pp. 1120–1131, 2005.
- [57] G. Bracha, "Asynchronous Byzantine agreement protocols," *Information and Computation*, vol. 75, no. 2, pp. 130–143, 1987.
- [58] M. K. Aguilera, N. Ben-David, R. Guerraoui, D. Papuc, A. Xyglis, and I. Zlotchi, "Frugal Byzantine computing," *International Symposium on Distributed Computing (DISC)*, 2021.
- [59] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman, "All you need is DAG," in *ACM Symposium on Principles of Distributed Computing (PODC)*, 2021, pp. 165–175.
- [60] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, "Narwhal and tusk: a DAG-based mempool and efficient BFT consensus," in *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2022, pp. 34–50.
- [61] O. Oleksenko, B. Trach, R. Krahn, M. Silberstein, and C. Fetzer, "Varys: Protecting {SGX} enclaves from practical {Side-Channel} attacks," in *Usenix Annual Technical Conference (USENIX ATC)*, 2018, pp. 227–240.

APPENDIX A

LITERATURE REVIEW

BFT with hardware assistance. In traditional quorum-based BFT [44] with $3f+1$ processes, a supermajority quorum of $2f+1$ ensures that the intersection of any two quorums includes at least one correct process, preventing conflicts [45]. By incorporating trusted hardware that restricts malicious behavior, a quorum of $f+1$ with $2f+1$ total processes is enough to prevent conflicting quorums. With the non-equivocation mechanism and digital signatures, a crash-fault protocol can be compiled into a Byzantine-fault protocol with the same amount of faulty processes in the asynchronous communication model [46], [47]. As a result, protocols with trusted hardware assistance

can tolerate half of the processes being malicious [23], [24], [30], [31].

PBFT with proactive recovery marked the first instance of a BFT protocol integrating a locally-trusted component, a synchronous counter [48]. However, it was later proven that implementing resilient proactive recovery is impossible without a proper architectural framework for these trusted components [49]. Correia et al. first introduced the Trusted Timely Computing Base (TTCB), a tamper-proof distributed oracle for intrusion-tolerant atomic multicast service with $2f+1$ processes [50]. Following this, Attested Append-Only Memory (A2M) offers trusted, undeniable, ordered logs to prevent equivocation, enhanced in PBFT-EA for local device implementation [33]. TrInc, a lightweight trusted incrementer, provides a more versatile solution for large-scale systems, further abstracted in Hybster's TrInx for diverse certificate generation [31], [30]. Subsequently, MinBFT and EBAWA utilize the "Unique Sequential Identifier Generator" (USIG) for message certification, reducing communication rounds and ensuring non-equivocation [23], [51]. CheapBFT introduces an FPGA-based subsystem for unique message certification, allowing $f+1$ active processes for optimistic operations [52]. FastBFT then shares features with CheapBFT but adds a hardware-based secret-sharing mechanism for reduced message complexity [53]. More recently, Damysus enhances HotStuff protocol's resilience and communication overhead by combining trusted counters with secure storage [24]. HotStuff-M [35] integrates trusted hardware with expander graph techniques, enhancing resiliency without raising communication complexity. It proposes a validated ABA as a trusted asynchronous BFT protocol with optimal resilience. However, it adopts the external validity property defined in CKPS without offering a reduction from ACS to it.

ACS paradigms. First introduced in the context of asynchronous multi-party computation (MPC), ACS is typically formed by two modules consisting of reliable broadcast (RBC) and asynchronous binary agreement (ABA) [54], which are later implemented in the classic BKR paradigm. BKR incorporates a broadcast instance and an agreement instance for each party in the network [15] and is followed by a series of works for further refinement [2], [3], [4]. The CKPS paradigm replaces ABA with MVBA, resolving ACS by agreeing on multivalued inputs [6]. Subsequent works inspired by MVBA introduced various paradigms, refining MVBA to match the functionalities and benefits of BKR ACS models [18], [20], [22]. A comprehensive comparison was conducted between the two ACS paradigms, ABA-based and MVBA-based, referred to as BKR and CKPS respectively in their work [16]. The work shows that the ABA-based ACS framework is information-theoretically secure and achieves quantum safety with $\mathcal{O}(\log n)$ running time. In comparison, the MVBA-based framework is only computationally secure and relies on less well-established cryptographic pairing assumptions; it has an $\mathcal{O}(1)$ running time but with a large hidden constant.

Asynchronous Byzantine agreements. HoneyBadgerBFT is

a practical asynchronous BFT protocol that ensures liveness without relying on any timing assumptions. The expected runtime of HoneyBadgerBFT can be reduced through the execution of parallel instances, although this approach may impact throughput [2]. BEAT family enhances HoneyBadgerBFT by introducing five asynchronous BFT protocols with a modular design. This improvement offers greater versatility for various scenarios, providing more performance trade-off options [3]. The subsequent work WaterBear further designs and implements a family of practical asynchronous Byzantine Fault Tolerance (BFT) protocols, matching the security guarantees of their partially synchronous counterparts [55]. Dumbo BFT enhances the efficiency and scalability of HoneyBadgerBFT by limiting the number of concurrent consensus instances and shifting BA from binary to a multi-valued format [19]. An enhanced version, SpeedingDumbo, introduces a linear broadcast protocol and a compact consensus protocol to optimize complexity [22]. PACE is designed for asynchronous BFT protocols to alleviate the performance bottleneck in parallel BFT instances, which can delay or yield an empty final consensus outcome. It proposes an enhanced asynchronous binary BFT protocol with a reduced number of steps per round and a more efficient common coin algorithm [16]. FIN is an asynchronous BFT protocol to deliver a signature-free ACS with optimal time complexity [14]. Some works [56] alternatively employ vector outputs to support ACS (characterized as the vector consensus problem).

Broadcast protocols. Reliable broadcast protocols are predominantly used for their message delivery properties in asynchronous settings. Bracha's double-echo RBC protocol has $\mathcal{O}(n^2)$ message complexity due to the all-to-all communication rounds property [57]. While efforts have been made to integrate trusted hardware into double-echo RBC, they still necessitate one round of all-to-all communication, and the message complexity remains $\mathcal{O}(n^2)$ [58]. Alternatively, provable broadcast has been explored in pipelined BFT protocols for its linear message complexity [34], [35], [24].

DAG-based BFT. For concurrent transaction processing, directed acyclic graph (DAG)-based BFT protocols take a different approach by providing a unique data structure for transactions as well as their causal relations [59], [60]. Transactions can be issued at the same time as long as their causal relations are clear, which is concluded by the sequence of rounds in which the transactions are issued. For this reason, transactions can be issued in parallel if they are causally unrelated. The "super-inclusive" interpretation rule for the DAG structure can include and order all the historic transactions linked to a primary transaction. DAG-rider proposes the first DAG-based asynchronous BFT protocol where each process generates a transaction (in a block) and links all the previously received blocks as its causal predecessors. When interpreting the DAG, the DAG-rider considers all the transactions linked to a primary block valid [59]. In a later publication, a DAG-based mempool Narwhal separates the blocks and their credentials to reduce communication overhead and its interpretation protocol

Tusk improves the DAG-rider with faster termination [60].

APPENDIX B DEFERRED PROOFS

Lemma 1 (Janus-BA-agreement). *A BA instance x in the JANUS protocol achieves BA-agreement.*

Proof. We proceed by contradiction. Let the correct process P_i decide w for instance s at round r_i and the correct process P_j decide $w' \neq w$ for instance x . Let r_j the round when P_j decides w' . We consider two cases:

- *Case-1:* $r_j = r_i$
- *Case-2:* $r_j \neq r_i$

In *Case-1*, it implies that *CommonCoin* has different results for P_i and P_j at the same round, a contradiction.

In *Case-2*, we consider $r_j > r_i$ w.l.o.g. and first consider $w = 1$. In round r_i , i received $f + 1$ *Vote* messages where $v[x] = 1$ from distinct processes. This means there are at least $f + 1$ processes that have Certified or Voted status, and up to f processes with the Void or Endorsed status. In r_j , P_j received $f + 1$ *Vote* messages where $v[x] = 0$ from distinct processes. However, there are only up to f processes that have Void able to send a *Vote* message where $v[x] = 0$, a contradiction.

Finally, consider $w = 0$. In round r_i , P_i received $f + 1$ *Vote* messages where $v[x] = 0'$ from distinct processes. This means there are at least $f + 1$ processes that have the Zeros status, and only up to f processes that have the Endorsed status. There are consequently no processes with Certified as $f + 1$ processes with Endorsed are required. In r_j , P_j received $f + 1$ *Vote* messages where $v[x] = 1$ from distinct processes. However, there are no processes with Certified, able to send a *Vote* message where $v[x] = 1$, a contradiction. $\square$

Lemma 2. *If all processes start round r with Certified for instance x and *CommonCoin* is 1, a correct process decides 1.*

Proof (Lemma 2). Processes are only able to send *Vote* messages in round r where $v[x] = 1$. Upon receiving n -f *Vote* messages, a correct process P_i decides 1 as *CommonCoin* is 1 by assumption. $\square$

Lemma 3. *If all correct processes start round r with Endorsed for instance x and *CommonCoin* is 1, a correct process decides 1 in the next round $r + 1$ if *CommonCoin* for $r + 1$ is also 1.*

Proof (Lemma 3). Correct processes are only able to send *Vote* messages in round r where $v[x] = \text{null}$. Upon receiving n -f distinct *Vote* messages, at least one has $v[x] = \text{null}$ and a *Vote* message from P_x (the sender) is attached. As a result, any process updates its state to Certified. All processes then start round $r + 1$ with the Certified status where *CommonCoin* is 1 and by Lemma 2, a correct process decides 1. $\square$

Lemma 4 (Janus-BA-termination). *A BA instance x in the JANUS protocol achieves BA-termination.*

Proof. The proof is divided into two parts: (i) In each round, correct processes eventually proceed to the next round, and (ii) a correct process at round r has decided after round $r + 2$ with probability at least $1/8$. As $\liminf \Pr[E_k] \leq \liminf (1 - \frac{1}{8})^k = 0$ for the event E_k that no correct process has decided at round k , a correct process eventually decides. $\square$

(i) *In each round, the correct processes eventually proceed to the next round.*

Proof. We prove this by induction.

Initialization: In round 1, each process receives a minimum of n -f transactions as there are at least n -f correct senders. Upon receiving n -f transactions, each correct process will create a *Vote* message, receive n -f *Vote* messages, and proceed to round 1.

Induction: In round k , each correct process broadcasts a valid *Vote* message. A correct process will receive n -f *Vote* messages, update its internal status in trusted hardware, produce a *Vote* message for round $k + 1$ and proceed to round $k + 1$, completing the induction. $\square$

(ii) *a correct process at round r has decided after round $r + 2$ with probability at least $1/8$.*

Proof. Consider three cases at round r , where n_1 is the number of processes that have the Certified state:

- *Case-1:* $n_1 \geq f + 1$
- *Case-2:* $f + 1 > n_1 \geq 1$
- *Case-3:* $n_1 = 0$

In *Case-1*, we proceed by contradiction. Assume a correct process P_i is unable to terminate for instance x indefinitely in this case (the possibility is 0). Upon receiving n -f *Vote* messages, P_i must have received at least one *Vote* message where $v[x] = 1$ because there are no less than $f + 1$ processes voting 1. Hence, processes update their state to Certified and can only vote 1 in the next round, and all correct processes vote 1. Consequently, by Lemma 2, P_i will decide in the next round if *CommonCoin* is 1 which happens with probability $1/2$, a contradiction.

In *Case-2*, a process voting 1 with a Certified state implies that at least $f + 1$ processes have the Endorsed state to form a QC. As a result, no *Vote* messages where $v[x] = 0'$ can be created. A process P_i , upon receiving n -f *Vote* messages, receives at least one *Vote* message where $v[x] = \text{null}$ with a *Vote* message from P_x (the sender) attached. If *CommonCoin* is 1 which happens with probability $1/2$, all correct processes then have state Endorsed. By Lemma 3, a correct process will therefore decide in round $r + 2$ if *CommonCoin* is 1 in rounds $r, r + 1$, and $r + 2$. We have a $1/8$ probability of deciding in case 2, a contradiction.

In *Case-3*, if there is no process with the Certified state, all processes either vote 0 or null. Upon receiving n -f *Vote* messages, processes have either only *Vote* messages with $v[x] = 0$ or have at least one *Vote* message with $v[x] = \text{null}$. Let $\mathbb{Q}$ be the set of processes that have received n -f *Vote* messages with $v[x] = 0$ and $\mathbb{P}$ the set of remaining processes.

If $Q \geq f + 1$, there is a $1/2$ chance that *CommonCoin* is 0 and nodes in Q enter round $r + 1$ with Zeros. Upon receiving $n-f$ *Vote* messages in round $r + 1$, there is at least one *Vote* message with $v[x] = 0'$ and all processes update their state to Zeros if *CommonCoin* is 0. In round $r + 2$, all processes can only send *Vote* message with $v[x] = 0'$, and a correct process will decide 0 upon receiving $n-f$ *Vote* messages if *CommonCoin* is 0. *CommonCoin* is 0 in rounds $r, r + 1, r + 2$ with probability $1/8$, a contradiction.

If $P \geq f + 1$, there is a $1/2$ chance that *CommonCoin* is 1 and processes in P enter round $r + 1$ with Endorsed. Upon receiving $n-f$ *Vote* messages in round $r + 1$, there is at least one *Vote* message with $v[x] = \text{null}$ and all processes update their state to Endorsed if *CommonCoin* is 1. By Lemma 3, a correct process will decide in round $r + 2$ if *CommonCoin* is 1 in round $r + 1$ and $r + 2$. We therefore have a $1/8$ probability of deciding, a contradiction. $\square$

Lemma 5. A BA instance x in JANUS achieves BA-validity.

Proof. We proceed by contradiction in two cases:

- Case 1: A correct process P_i decides 0 and every correct process has received the proposal for x , inputting 1 to x .
- Case 2: P_i decides 1 and every correct process has not received the proposal for x , inputting 0 to x .

In *Case-1*, P_i has received $n - f$ *Vote* messages with $v[x] = 0'$, implying that a correct process P_j has sent a *Vote* with $v[x] = 0'$. P_j therefore has the state Zeros, implying that P_j has either received $n - f$ *Vote* with $v[x] = 0$ or received a *Vote* with $v[x] = 0'$ from another process P_x . This is a contradiction as there must be at least $n - f$ processes, including at least one correct process, initially sending a *Vote* messages with $v[x] = 0$, but all correct processes receive the proposal for x by assumption.

In *Case-2*, P_i has received $n - f$ *Vote* messages with $v[x] = 1$, implying that a correct process P_j has sent a *Vote* messages with $v[x] = 1$. P_j therefore has the state Endorsed, implying that P_j has either received a *Cert* message for instance x or received a *Vote* messages with $v[x] = 1$ from another process P_x . This is a contradiction as there must be an initial process P_x that received a *Cert* message x , requiring that at least $n - f$ processes, including at least one correct process, receive the proposal for x . $\square$

Theorem (JANUS BA). A BA instance x in the JANUS protocol achieves BA-agreement, BA-termination, BA-validity.

Proof (Theorem 1). The claim follows Lemmas 1, 4 and 5. $\square$

Theorem (JANUS ACS). JANUS guarantees ACS-validity, ACS-agreement, and ACS-totality.

Proof (Theorem 2). ACS-agreement is derived from BA-agreement in Theorem 1 since a correct process decides for every instance in the ACS-output set $\mathbb{S}$.

ACS-totality is established from BA-termination in Theorem 1 as correct nodes eventually decide for all instances, leading to the output of the set of decisions $\mathbb{S}$.

To prove ACS-validity, we proceed by contradiction. Let $\mathbb{S}$ be the output at all correct nodes, such that it contains the input of less than $n - f$ processes. At least $n - f$ instances, which make up the set $\mathbb{X}$, started with at least 1 process at the Certified state, implying that at least $n-f$ processes have the Endorsed state for these instances.

A process P_i , upon receiving $n-f$ *Vote* messages, received at least one *Vote* message where $v[x] = \text{null}$ with a *Vote* message from P_x (the sender) attached. If *CommonCoin* was 1 for the first round, all processes started the next round with Endorsed and sent a *Vote* message where $v[x] = \text{null}$ for all instances $x \in \mathbb{X}$.

If *CommonCoin* was 0, all processes kept the state Endorsed as they only received *Vote* message where $v[x] = \text{null}$ for all instances $x \in \mathbb{X}$. The process proceeded to the next round and kept entering the next round with the state Endorsed if *CommonCoin* was still 0 in subsequent rounds. Once *CommonCoin* was 1, processes decided for every instance $x \in \mathbb{X}$ as shown by Lemma 3. Therefore, there are $n - f$ decided instances included in the final output $\mathbb{S}$, referring to the input of at least $n - 2f$ correct processes, a contradiction. $\square$

APPENDIX C

SECURITY ANALYSIS OF JANUS UNDER ATTACKS

A. Side Channel Attack

We consider the cache timing attack and controlled-channel attack as they are the most practical to be used to attack enclaves [61].

For cache timing attacks to work in Janus, the faulty process should be able to set its cache to a predefined state, let its trust hardware interact with the cache for some time, and then read from the cache again to determine which parts were used by the TEE. Nonetheless, the trusted memory is cached in SGX's Enclave Page Cache (EPC) and it prevents access from other software, including the operating system, from accessing the cached enclave data. The protection is guaranteed by the Memory Encryption Engine with encryption methods. In the worst case, even if the faulty host could access the memory bus, it would not be able to read the plaintext data.

For cache timing attacks to work in Janus, the faulty process should be able to observe the memory access pattern to extract sensitive information. Intel SGX has a dynamic memory management function and cache partitioning to mitigate this attack by allowing enclaves to allocate and deallocate memory at runtime and distributing the cache usage of the host process.

Although we acknowledge that there have been successful cases where applications running in SGX were compromised by side-channel attacks, Janus' use of trusted memory is primarily aimed at protecting information from arbitrary alteration rather than ensuring confidentiality from its host process. While successfully launching side-channel attacks against Janus is challenging, Janus would not suffer from any

operational errors anyway if the internal states were revealed to its faulty host in the worst-case scenario.

B. Iago Attack

To launch Iago attacks in Janus via the preconfigured interfaces, a faulty process would need to manipulate interface return values to compromise its internal states recorded within SGX enclaves. However, the inputs and outputs via Janus' interfaces are signed messages only. We believe that forging a signed message with a valid signature is practically impossible. Additionally, the inputs and outputs to Janus' interfaces are predetermined, and non-compliant messages, even if they have valid signatures, will not be accepted if the attacker tries inputting outdated messages. If the attacks attempt to manipulate the system call to directly read or write the data stored in the underlying cache, the Enclave Page Cache (EPC) can prevent such malicious behaviour.